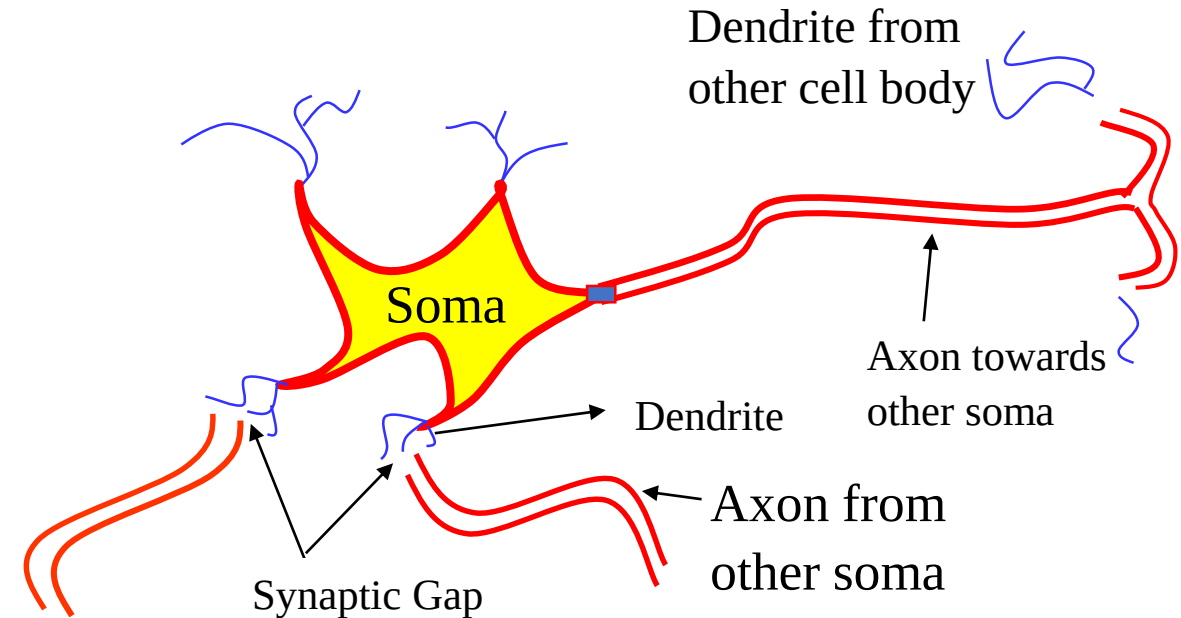
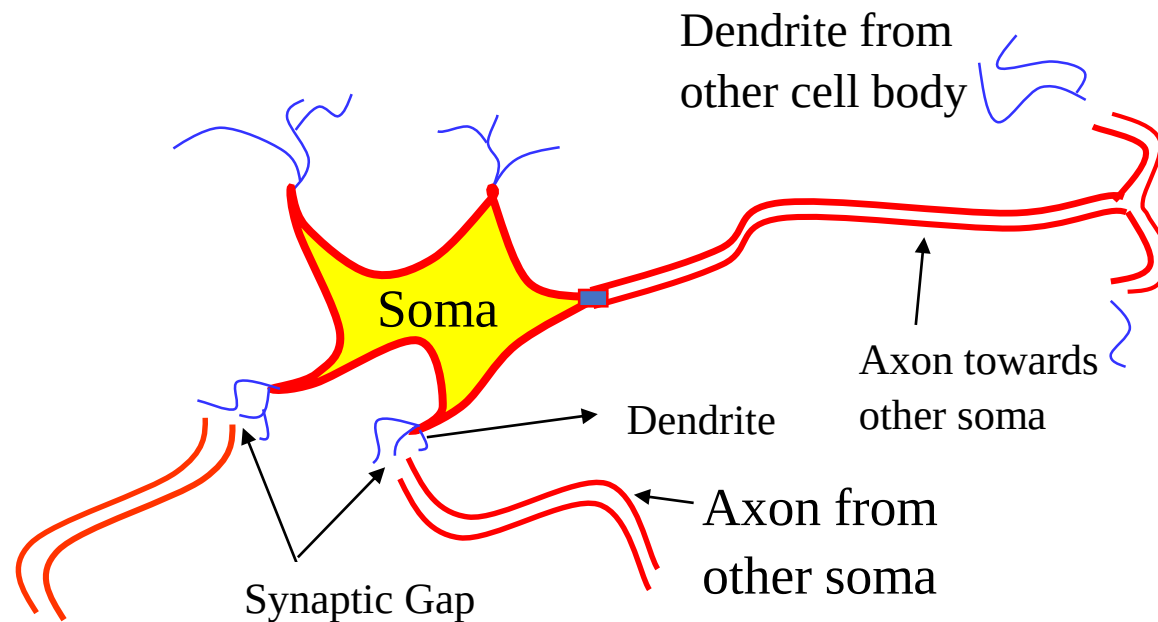


Artificial Neural Network

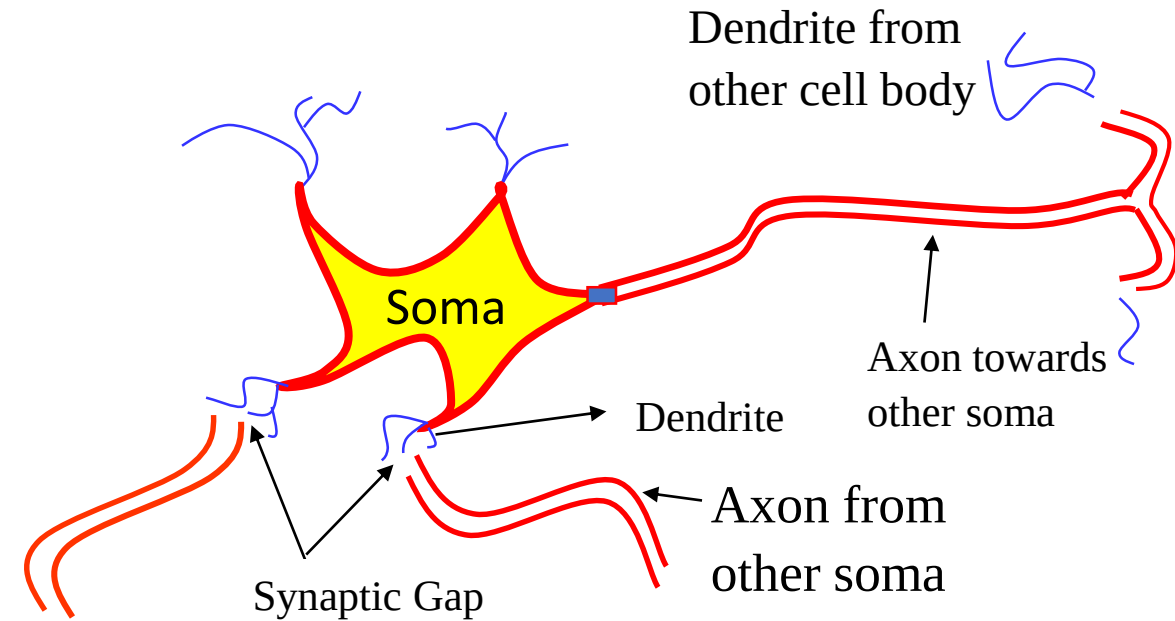
- ✓ Artificial Neural Network (ANN) works based on working principle of biological neuron consists of three main components: **dendrites**, **soma** and **axon**.
- ✓ The **soma** or the cell body connected to several **dendrites** through which it receives signal (electrical impulses).
- ✓ In ANN a node resembles to soma and dendrites are the input connecting lines of a node.



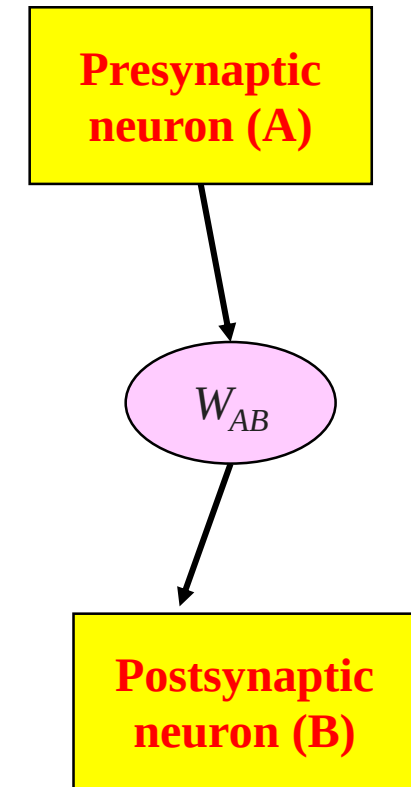
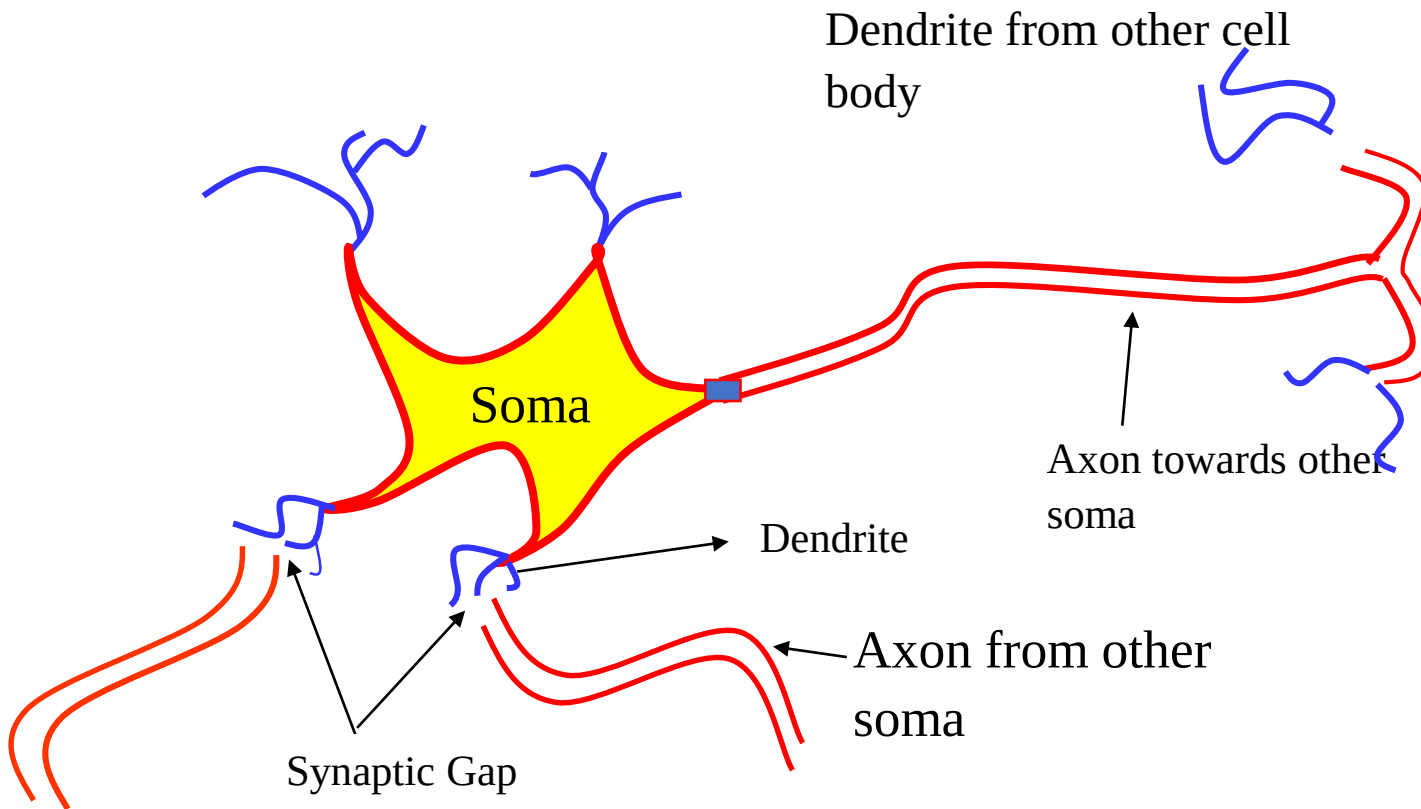
- ✓ The received signal at soma (from dendrites) are summed up and when the combined signal exceeds a threshold then the soma fires and transmits its signal to other soma using its output link called **axon** as shown in fig.1.
- ✓ The phenomenon of firing of neuron cell is equivalent **activation function** (for example sigmoid function) of ANN, where a node sends its signal to next node if the weighted sum of output signal exceeds a threshold.



- ✓ There is a gap between axon and dendrites called **synaptic gap** shown in fig.1, which resembles to an open circuit of flow of electrical signal.
- ✓ The transmission of electrical signal through this gap is done by chemical process which modifies signal (for example change in amplitude, phase, frequency etc.) and the phenomenon is represented by a **weighting factor** in ANN.

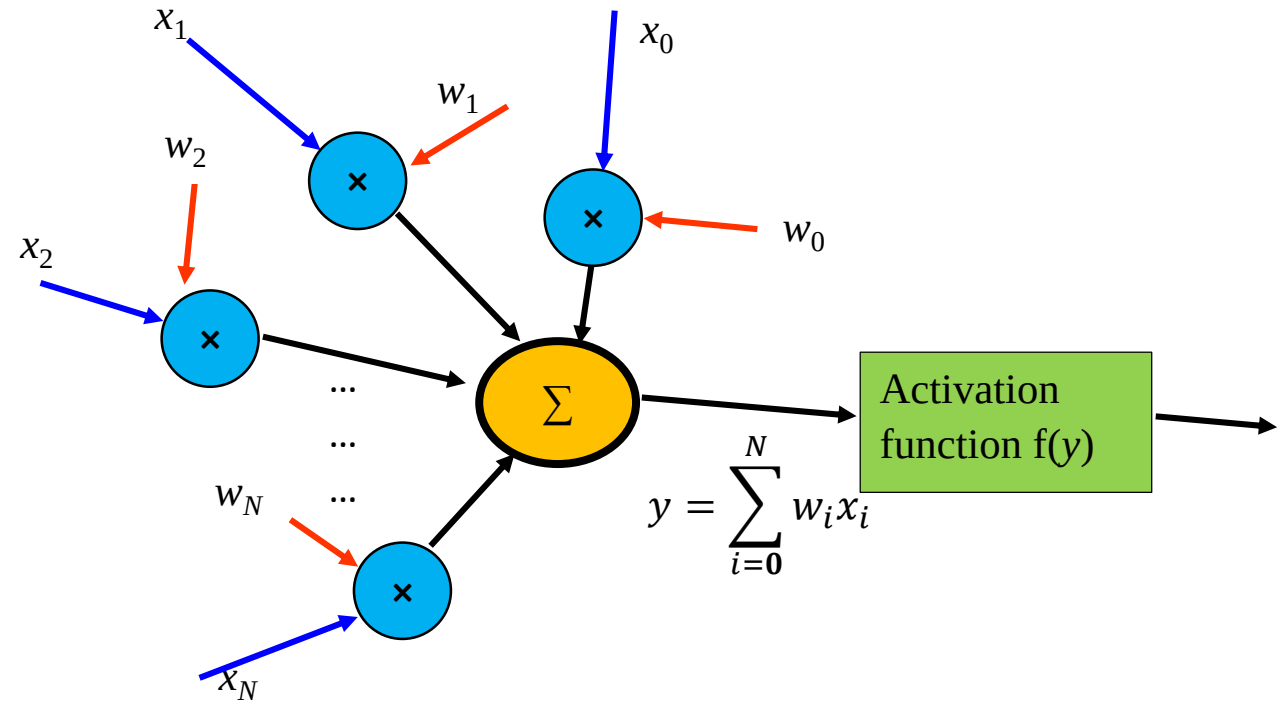


- ✓ When a neuron A transmits signal to its adjacent neuron B through the synaptic gap then the transmitting neuron A is called **presynaptic neuron** and received neuron B is called **postsynaptic neuron**. The strength of the received signal is governed by the synaptic gap and modeled as the weighting factor W_{AB} in ANN.



Modeling of Artificial Neuron

- ✓ The behavior of biological neuron can be represented by equivalent circuit of fig.2, called **artificial neuron** or **processing element** (PE). The biological neuron receives signals (input of soma) through the dendrites are represented as: $x_1, x_2, x_3, \dots, x_N$ in the artificial neuron.
- ✓ The strength of i th synapse is represented by weighting factor w_i . Stronger the transmitted signal by a synapse larger the weighting factor and vice versa. We can also consider weight as the strength of synaptic connection between neurons. The total signal received by the soma is expressed as, $y = \sum_{i=0}^N w_i x_i$



Two basic types of networks are distinguished, considering the connectivity of the neurons in a network:

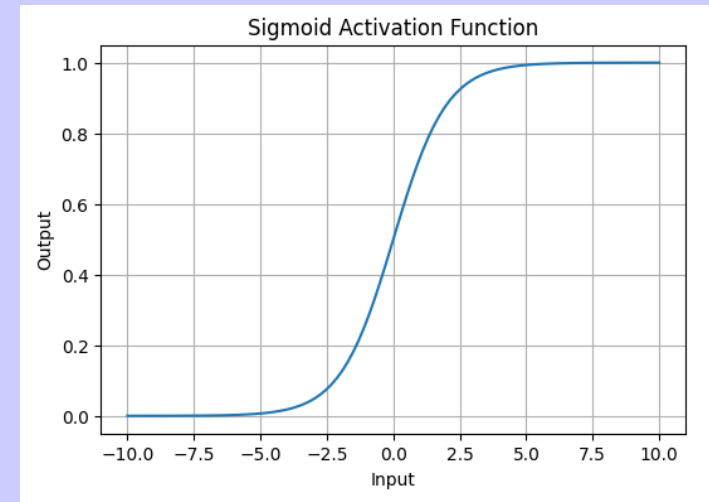
- ❖ Feed-forward network (for example Multilayer perceptron), no feed back of signal.
- ❖ Recurrent network or feedback network (for example Hopfield networks)

✓ If the sum of the signal at soma exceeds a threshold τ , then the then the soma fires and transmits signal to next neuron, otherwise no signal is conveyed by the neuron. The phenomenon is represented by **activation function** $f(y)$. There are several types of activation function used in ANN and few of them are shown below.

(a) Step function: $f(y) = \begin{cases} 1, & \text{if } y > 0 \\ 0, & \text{if } y \leq 0 \end{cases}$

(b) Signum function: $f(y) = \begin{cases} +1, & \text{if } y > \tau \\ -1, & \text{if } y \leq \tau \end{cases}$

(c) Sigmoid function: $f(y) = \frac{1}{1+\exp(-\alpha y)}$, where α is the slope parameter.



Learning Algorithms of ANN

- ✓ Learning of any **adaptive system** means to adapt itself with any change in its environment (input signal of the system). The learning of ANN deals with adjustment of weights to minimize the error (difference between output of ANN and desired output).
- ✓ The error is reduced with every iteration of learning algorithm and the learning becomes complete when there is no change in error with additional iteration of observation. The performance of learning algorithm depends how fast the error (cost function) of the ANN reaches to minimum.

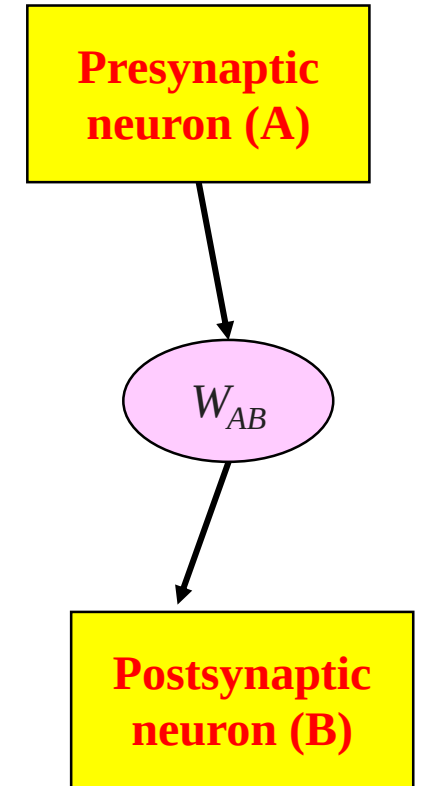
Few common learning techniques are:

- ✓ Hebbian Learning
- ✓ Supervised Learning
- ✓ Unsupervised learning
- ✓ Reinforced Learning

Hebbian Learning

The oldest learning technique of ANN is Hebbian learning, found from the classic work of neuropsychologist **Donald O. Hebb**. The most important quotation of the book, **The Organization of Behavior** (1949, p.62) by Hebb related to efficiency of biological neuron is found as:

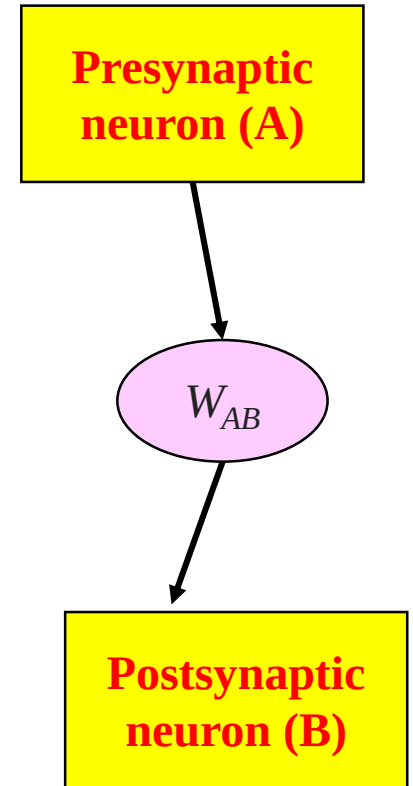
When an axon of cell A is near enough to excite a cell B and repeatedly or persistently takes part in firing it, some growth process or metabolic changes take place in one or both cells such that A's efficiency as one of the cells firing B, is increased.



Hebb's postulate (1949) was extended as:

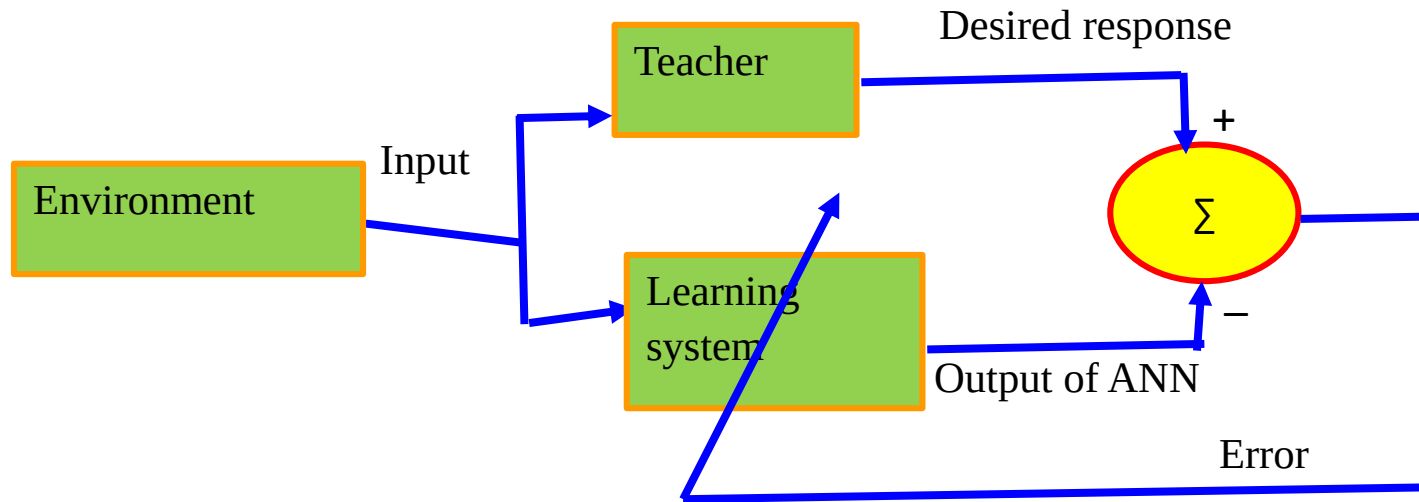
- ✓ If two neurons on either side of a synapse (connection) are activated simultaneously (or **synchronously**), then the strength of that synapse (weight of connection) is selectively increased.
- ✓ If two neurons on either side of a synapse (connection) are activated **asynchronously**, then that synapse (weight of connection) is selectively weakened or eliminated.

Above phenomenon is called **time dependent mechanism** of Hebbian learning.



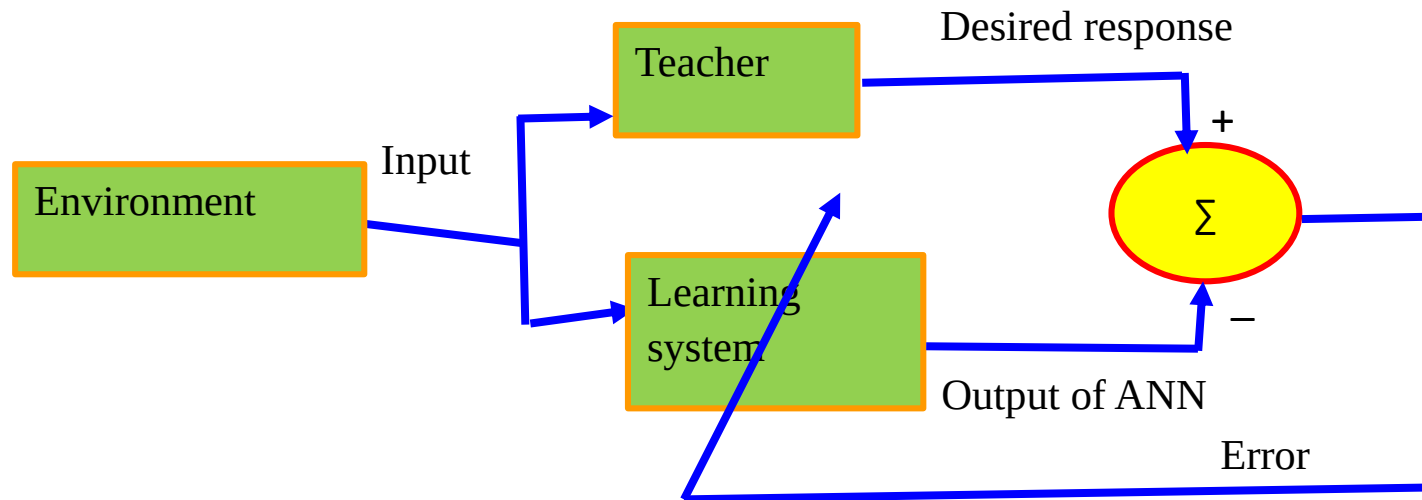
Supervised Learning

- ✓ The supervised learning is also called learning with teacher. The basic diagram of an ANN with supervised learning system is shown in fig. below. The supervised learning is a closed loop system where the environment (it provides input signal) is outside of the loop.



- ✓ The neuron does not know about the environment but the teacher has knowledge about the environment i.e. the teacher knows which input produces which output. When an input vector is applied in ANN the teacher produces the optimum result called desired response at the same time the ANN also produce an output which is the weighted sum of input elements.

- ✓ The difference between the output of ANN and desired response is called error. The ANN adjusts its weights to minimize the error. The weight adjustment is done step by step as if the ANN emulates the teacher.
- ✓ The environmental knowledge acquired by a teacher is transferred to the ANN through training (steps of weight adjustment algorithm). When the optimum situation is reached the ANN disconnects the teacher and deals with environment directly i.e. **shifted from training mode to test mode**.



Decision tree
Support Vector Machine
Linear Discriminant Analysis
Classification with Regression

Unsupervised learning

- ✓ Unsupervised learning deals with **unlabeled data** i.e. input data set without any **target value**. In this case the learning algorithm does not have any idea about environment to acquire desired value (optimum output value) from corresponding input vector. The learning algorithm segregate data from the internal properties of data for example similarity of dissimilarity of data points.
- ✓ During training session, the neural network received signal vector and based on output it separates the **stimulus (input signal like feature vector of image)** into categories. When input signal or stimulus is applied later it segregates the stimulus but if no class is found against the stimulus then it creates new class.

- ✓ Unsupervised learning does not have a teacher but need some guideline or algorithm is needed to convert the object (for object classification case object is converted to feature vector) to some numerical data relevant to its characteristics called features of the object.
- ✓ For example, in image clustering the features of the picture can be ‘profile of histogram of the image’, ‘shape of the image’, ‘spectrum of the image’, ‘properties of filtered image’, ‘statistical parameters of image matrix’, ‘properties from segmentation of image’, ‘geometry of special pixels’ texture of image etc.
- ✓ The ANN takes the feature vector of object as the stimulus instead of the object itself for classification purpose.

k-Means and FCM Clustering

Eigen Value and Singular Value Decomposition

Principal Component Analysis

Hidden Markov Model

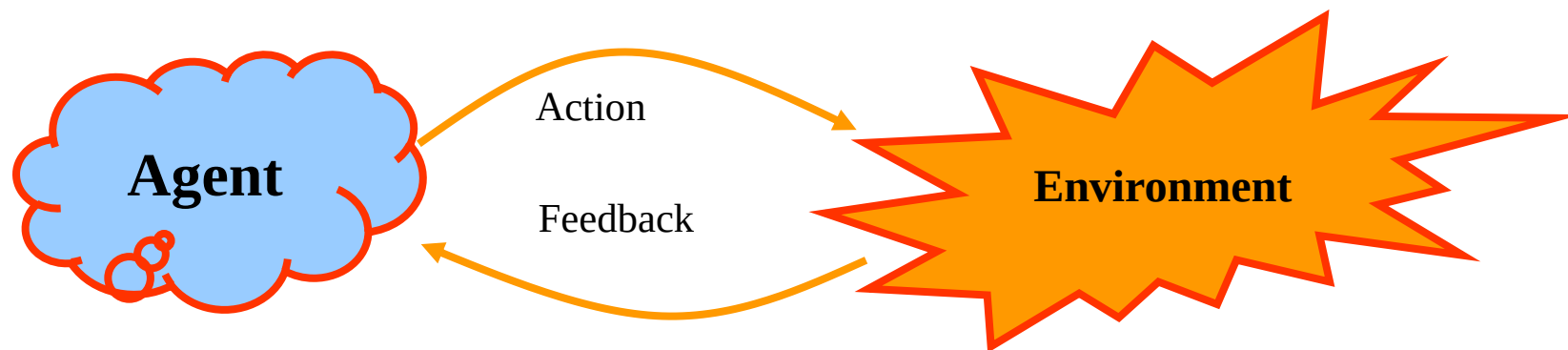
Reinforced Learning

- ✓ In this learning there is a teacher (like referee of a game), who only indicates the pass or fail status of the network. The network is in pass state when the target value is equal to the output value of the network otherwise it is in fail state is expressed as,

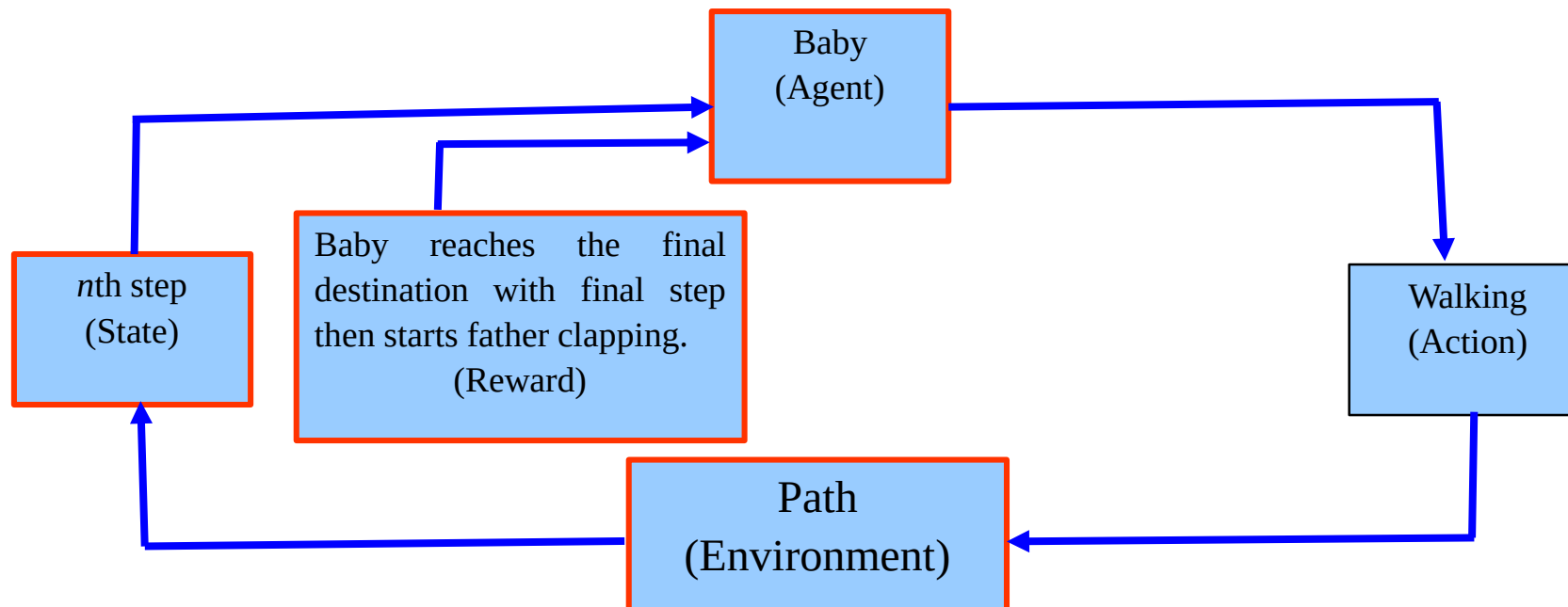
$$Status = \begin{cases} \text{if } error = 0, & \text{Network is in pass state} \\ \text{if } error \neq 0, & \text{Network is in fail state} \end{cases}$$

- ✓ When network is in fail state it re-adjusts its weights repeatedly until getting error signal of zero. Unlike supervised learning the teacher does not give any indication to the network that it is in right direction in changing weights.

- ✓ In RL we don't have any training data to learn the machine. The RL involves conversation or interaction between an **agent** and an **environment**, where environment provides a feedback for every action of an agent like figure below.
- ✓ In this learning an agent learns from an interactive environment (circumstances or physical world through which an agent moves) by trial and error, where agent gets feedback of its own actions (set of moves the agent can make) and he gathers experience about the environment like Markov chain of fig.4.
- ✓ The basic concept of this learning is given here in context of generalized machine learning (ML) technique instead of ANN.



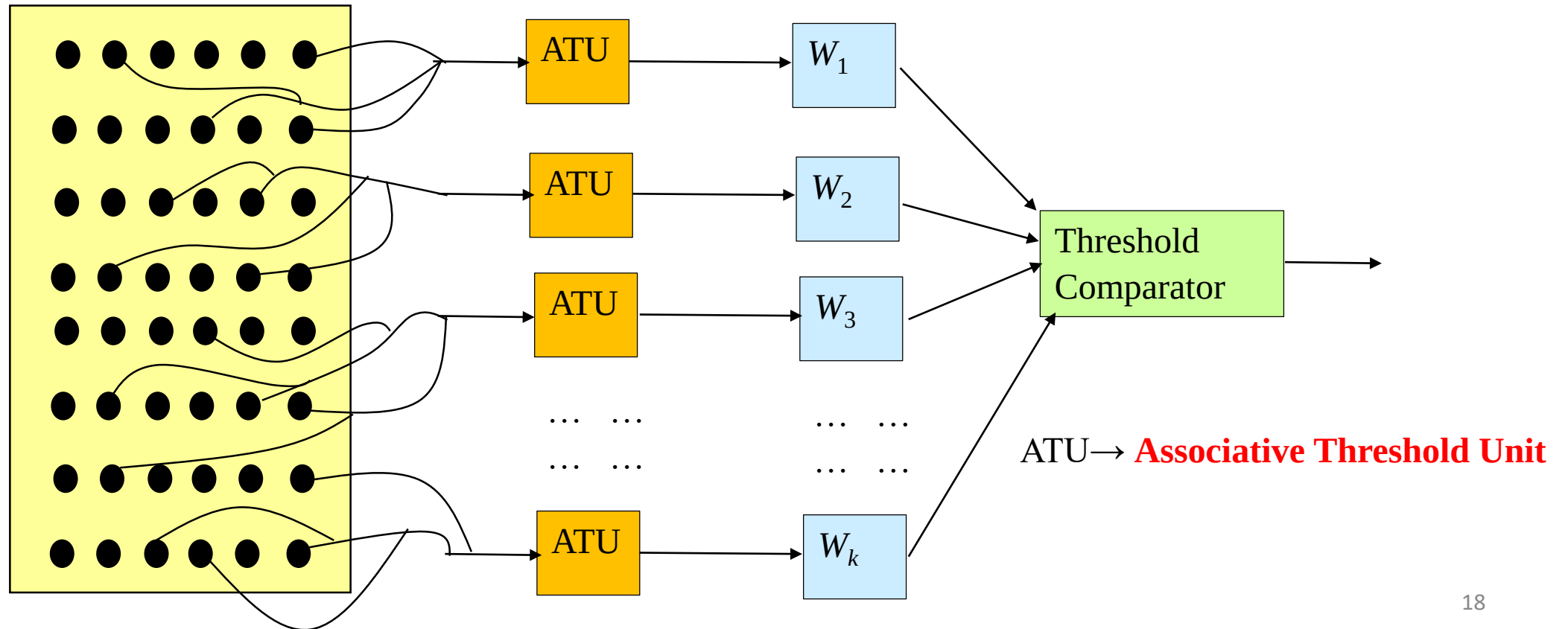
- ✓ An agent or **baby** starts **walking**, which is its action. The baby measures the success or failure in a given state (nth step) from the feedback of his environment called reward. Here the house is its environment.
- ✓ For example, if a baby puts correct steps towards his father then the baby gets claps called reward.



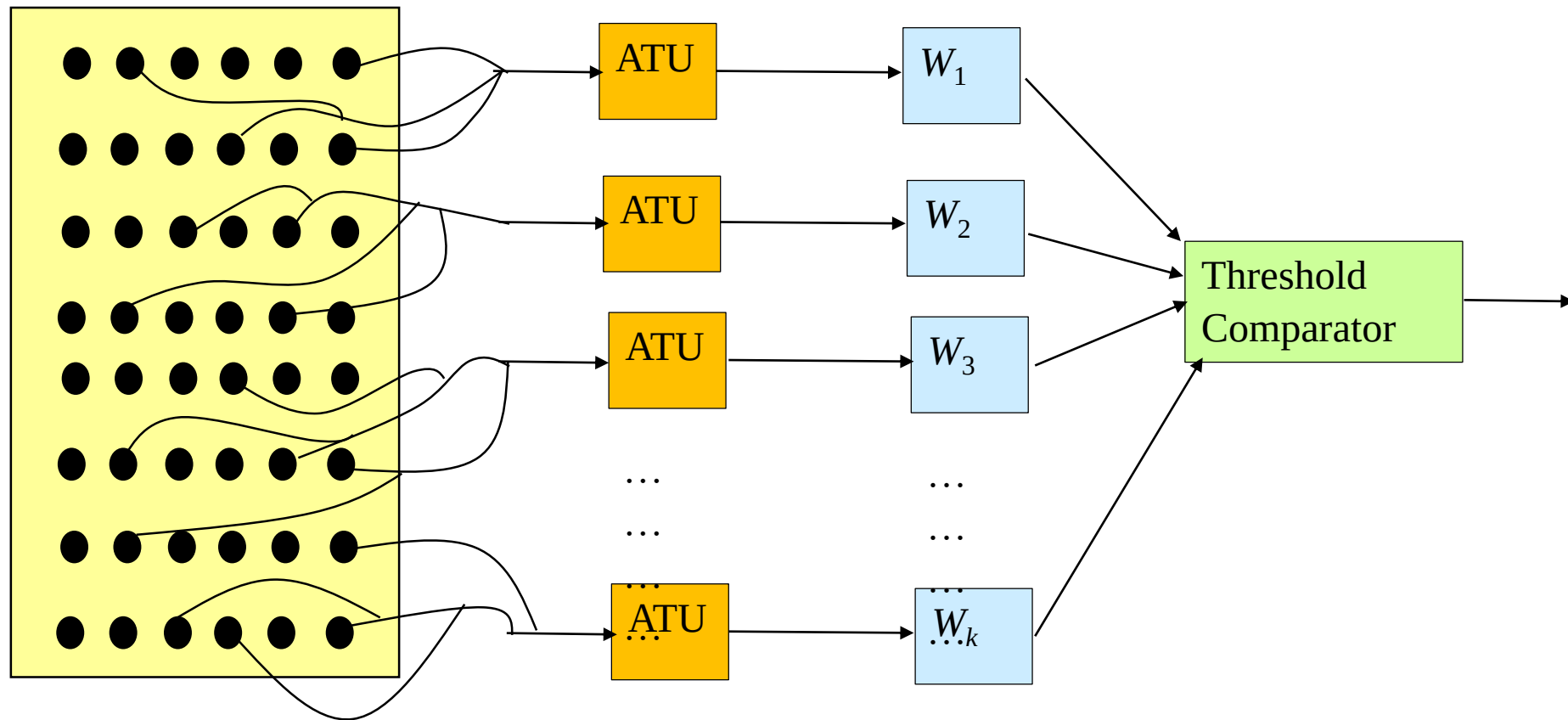
The reinforced learning (RL) uses different strategy in weight adjustment. In this learning two parameters are considered: time required per iteration and number of iterations to get the desired output.

Perceptron

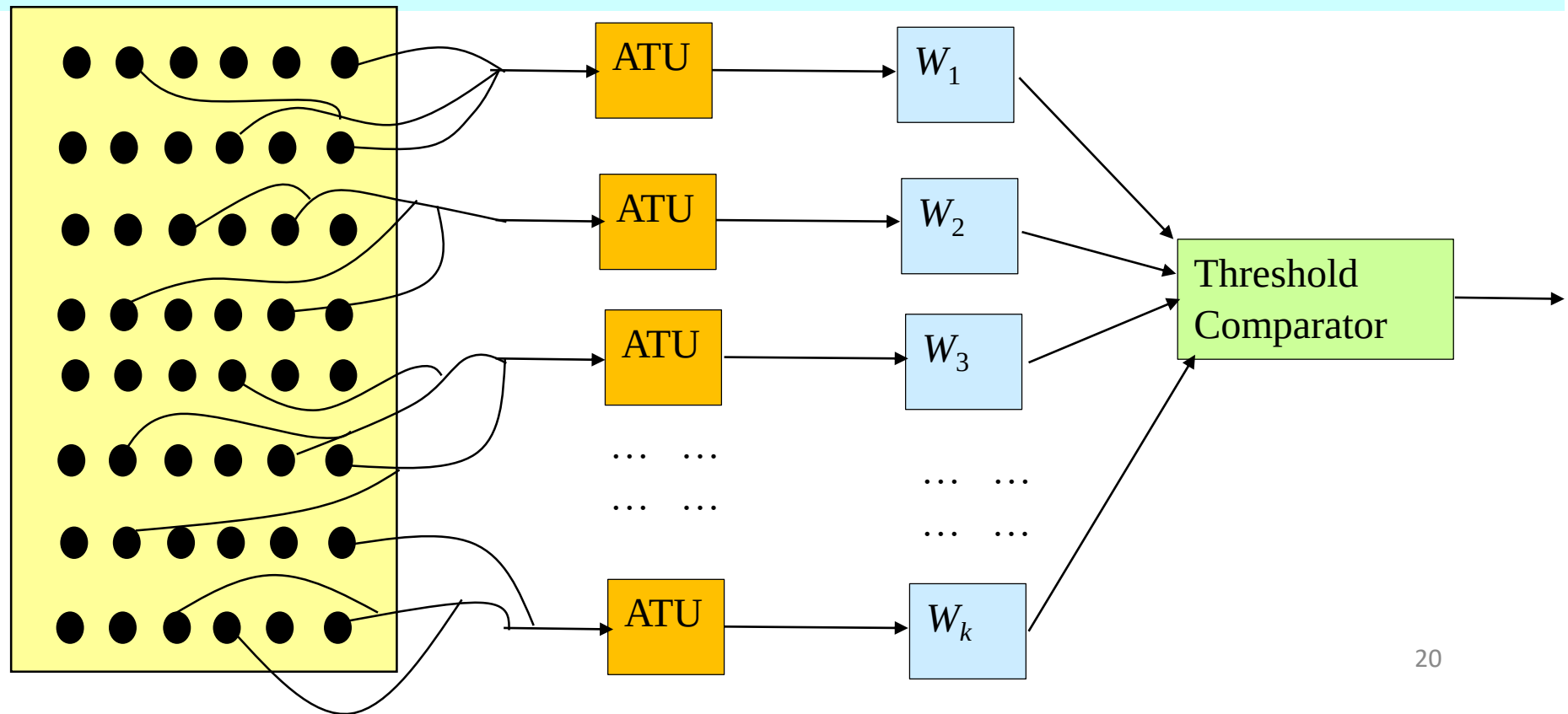
Perceptron is pattern recognition or classification device that are crude approximations neural networks. They make decisions about patterns by summing up evidence obtained from many small devices/sensors. A simplified perceptron is shown in fig.1, where the inputs to the system are through an array of sensors.



The sensors are randomly connected in groups to **associative threshold unit** (ATU), where sensor outputs are combined and added together. The light sensors produce an output voltage that is proportional to the light intensity striking them. If the combined outputs to an ATU unit exceeds some fixed threshold, the ATU unit executes and produce a binary output.

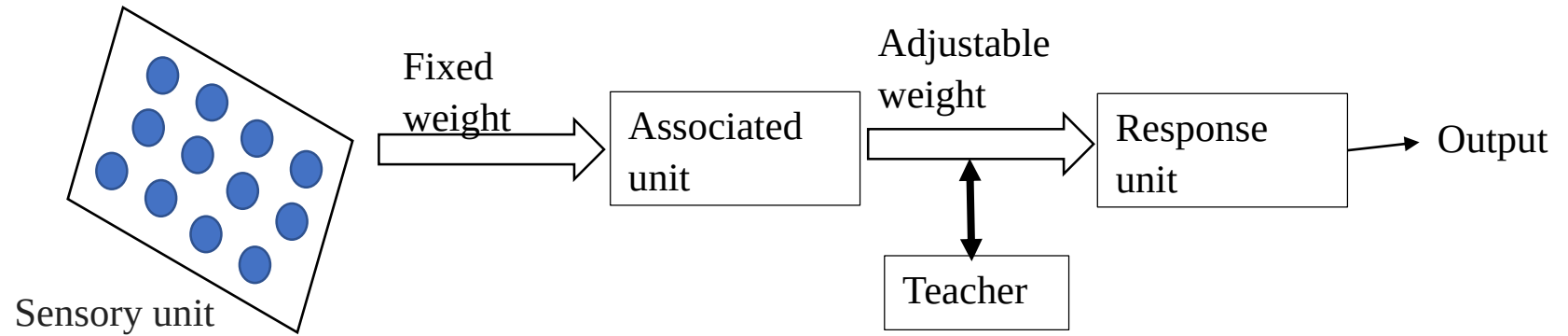


- ✓ The outputs of ATU are each multiplied by adjustable weight parameter w_i ($i = 1, 2, 3, \dots, k$) and the results added together in a terminal comparator unit. If the input to the comparator exceeds the threshold T , the perceptron produces the positive output (yes) i.e. data under class-1 else negative output (no) of class-2 (binary classification).
- ✓ All components of the system are fixed except the weights w_i which are adjusted through a **punishment-reward** process i.e. perceptron adjusts its weights only when output is incorrect. It does not improve weights after getting the correct results but supervised learning in ADALINE improves weights even after getting correct result (until error below a threshold).



Basic network of perceptron

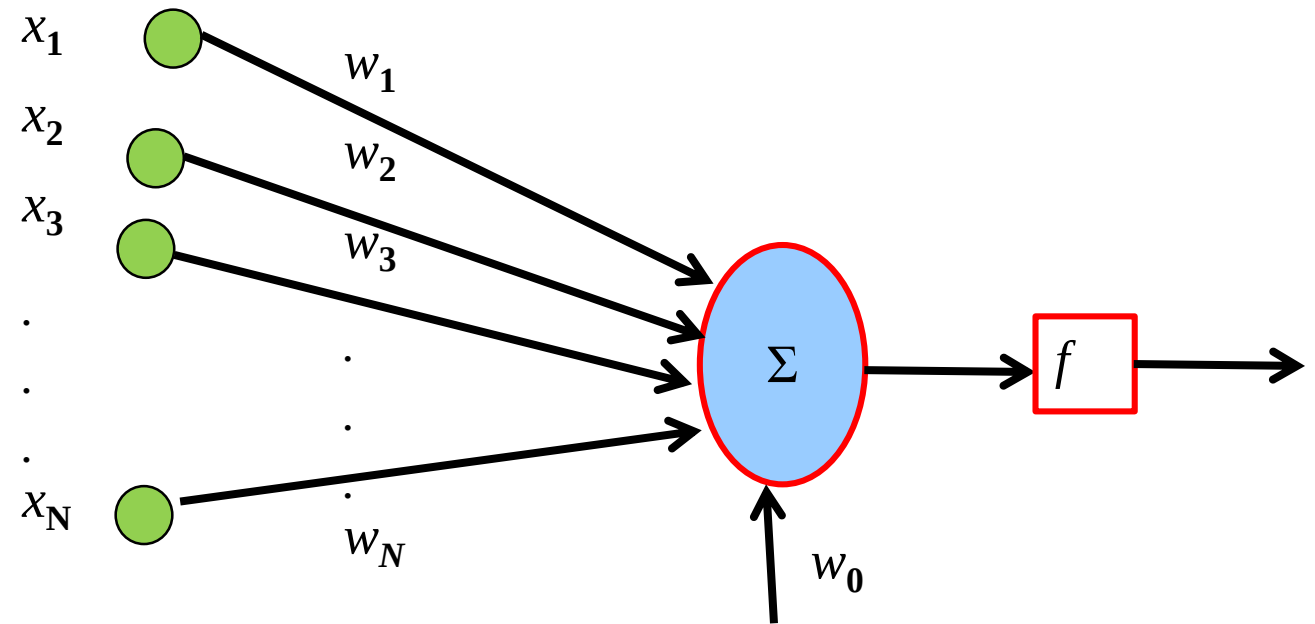
The original perceptron architecture has three layers of neurons: **sensory unit** (photodetector matrix), **associator units** (where learning is known i.e. weights are fixed) and a **response unit** (where training takes place to produce output) to form the computational model of retina of eye as shown in fig.1.



- ✓ The lighted picture impinges on the array of photodetector, which produces binary 1 or 0 depending on the intensity of light. If the light intensity is greater than threshold then photodetector produces electrical signal of logic 1, otherwise generates logic 0. The sensory unit is randomly connected to associated unit with fixed weights.
- ✓ The associated unit consists of a set of circuits called **feature demons** which produce features of object recognizable (or suitable) to retina. The weights between associated unit to response unit are adjusted by perceptron learning rule (better than Hebbian rule) to produce the final output response.

Recently different architecture is used for perceptron to model a single neuron. The entire scheme can be represented by diagram shown in fig.2, where the input vector $\mathbf{x} = [x_1 \ x_2 \ x_3 \ \dots \ x_N]^T$ is applied to the input of the network then each element is multiplied by corresponding weights w_i , $i = 1, 2, 3, \dots, N$. These weights are called synaptic weight or synapses. The biasing term w_0 is then summed up with the weighted sum.

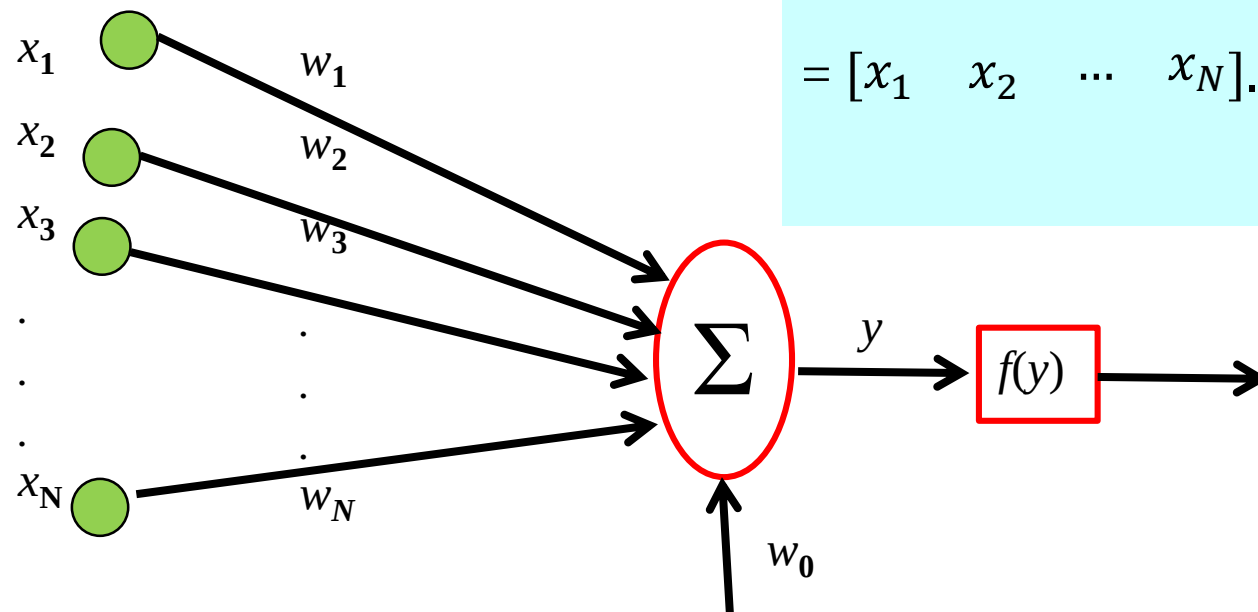
In ANN a single neuron, also referred to as the perceptron.



The output of adder is, $y = x_1w_1 + x_2w_2 + \dots + x_Nw_N + w_0 = \mathbf{w}^T\mathbf{x}$ and the result r goes to a non-linear device (decision with threshold) which is represented by an activation function $f(.)$. A common had limiter to classify the data point as,

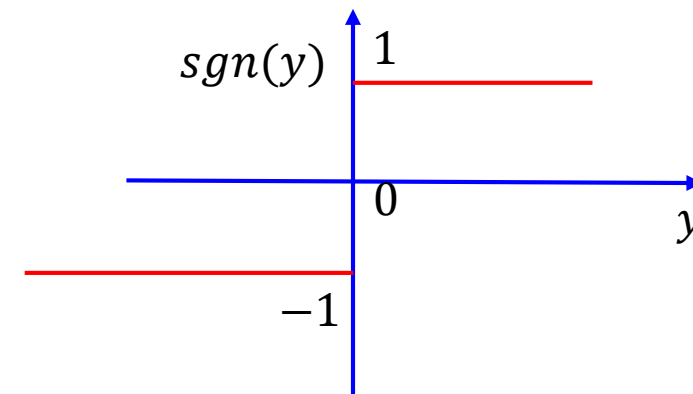
$$f(y) = \text{sgn}(y) = \begin{cases} +1, & \text{if } y \geq 0 \\ -1 & \text{if } y < 0 \end{cases}$$

Perceptron is used for binary classification like previous sections as: $\mathbf{w}^T\mathbf{x} < 0$ and $d_x = -1$ and $\mathbf{w}^T\mathbf{x} > 0$ and $d_x = 1$.



$$y = x_1w_1 + x_2w_2 + \dots + x_Nw_N + w_0$$

$$= [x_1 \quad x_2 \quad \dots \quad x_N] \cdot \begin{bmatrix} w_1 \\ w_2 \\ \vdots \\ w_N \end{bmatrix} + w_0 = [1 \quad x_1 \quad x_2 \quad \dots \quad x_N] \cdot \begin{bmatrix} w_0 \\ w_1 \\ w_2 \\ \vdots \\ w_N \end{bmatrix} = \mathbf{w}^T\mathbf{x}$$



The algorithm of perceptron in binary classification is given below.

1. Initialize the weights at instant $k = 0$ as: $w_0(0) = w_1(0) = w_2(0) = w_N(0) = 0$

2. The final output of the perceptron, $y(n) = \text{sgn}[\mathbf{w}^T(n) \cdot \mathbf{x}(n)]$,

where $\mathbf{w}(n) = [w_0(n) \ w_1(n) \ w_2(n) \ \dots \ w_N(n)]^T$, $\mathbf{x}(n) = [x_0(n) \ x_1(n) \ x_2(n) \ \dots \ x_N(n)]^T$,
the weight of bias $w_0(n) = b(n)$ and input of bias $x_0(n) = 1$.

3. Update the weight: $\mathbf{w}(n+1) = \mathbf{w}(n) + \eta[d(n) - y(n)] \cdot \mathbf{x}(n)$, η is called learning rate, which is constant, positive and less than unity.

Where the **desired response**, $d(n) = \begin{cases} +1 & \text{if } \mathbf{x}(n) \text{ belongs to class 1} \\ -1 & \text{if } \mathbf{x}(n) \text{ belongs to class 2} \end{cases}$

4. Go back to step 2 and continue until weight update stops

From above algorithm it is visualized that when $d(n) = y(n)$, there is no update of weights and algorithm stops for that input vector.

Example-1

Given input vector, $\mathbf{x} = [1 \ 1.2 \ 0 \ -0.5 \ 1.3]$, initial weight, $\mathbf{w} = [1 \ -1.6 \ 2 \ 0.35 \ 0.75]$, biasing $w_0 = -0.6$ and desired output $d = 1$.

Ans.

The output, $y(0) = \text{sgn}\{1*1+1.2*(-1.6)+0*2-0.5*0.35+1.3*0.75-0.6\} = \text{sgn}(-0.72) = -1$
Since $y(0) \neq d$, we need to change the weight.

$$\begin{aligned}\mathbf{w}(1) &= \mathbf{w}(0) + 0.8*[d(0)-y(0)]*\mathbf{x} = [1 \ -1.6 \ 2 \ 0.35 \ 0.75]^T + 0.8*(1-(-1))*[1 \ 1.2 \ 0 \ -0.5 \ 1.3]^T \\ &= [2.6 \ 0.32 \ 2 \ -0.45 \ 2.83]\end{aligned}$$

$$y(1) = \text{sgn}(1*2.6+1.2*0.32+0*2-0.5*(-0.45)+1.3*2.83-0.6) = \text{sgn}(6.28) = 1$$

Since $y(1) = d$, no need to change the weight.

ADALINE and Adaptive Algorithm

- ✓ The **ADALINE** is a single vector processing element (PE) like **adaptive digital filter** even very closed structure like perceptron. The acronym of ADALINE is **Adaptive Linear Neuron** or later **ADaptive LINear Element**. The basic structure of ADALINE consists of an adaptive linear combiner and bipolar function (also called thresholding function) as shown in fig.2.1.

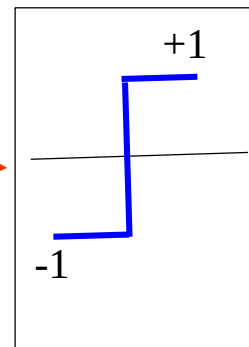
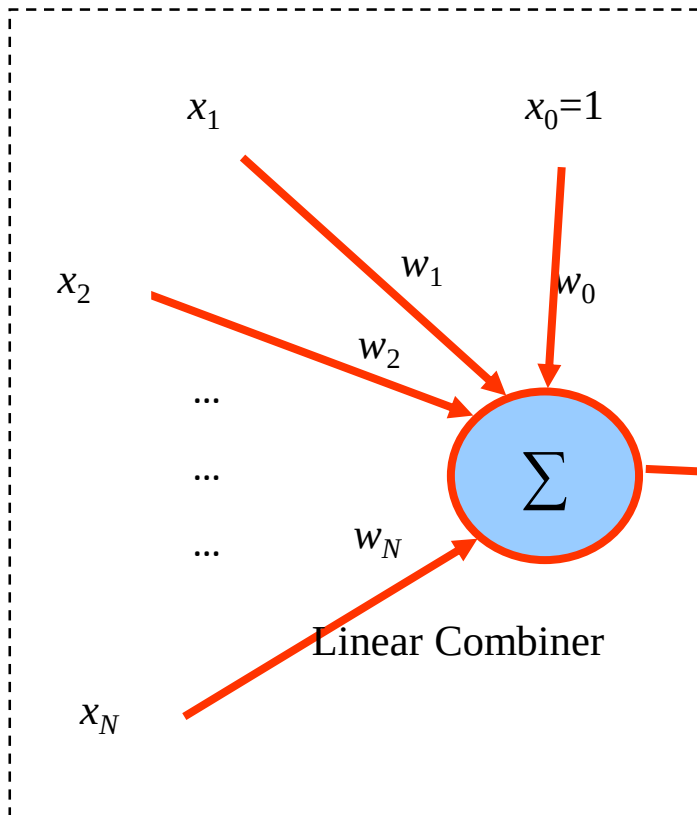
The parameters of ADALINE are:

Input signal vector, $\mathbf{X} = [x_1 \ x_2 \ x_3 \ \dots \ x_N]^T$

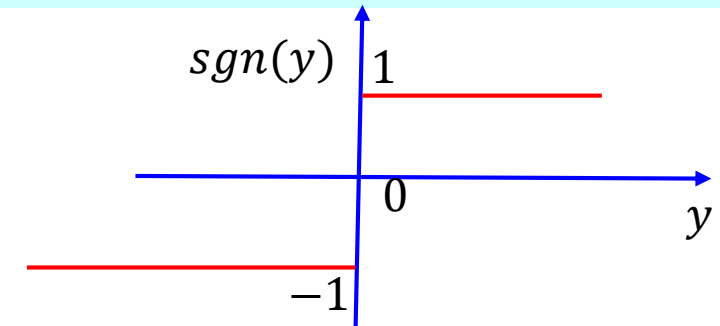
Weighting vector of input, $\mathbf{W} = [w_1 \ w_2 \ w_3 \ \dots \ w_N]^T$

Bias weight, w_0 and bias input, $x_0 = 1$.

If the output $y \geq 0$ then the output of the ADALINE, $O = f(y) = +1$ and when $y < 0$ then the output of the ADALINE, $O = f(y) = -1$.



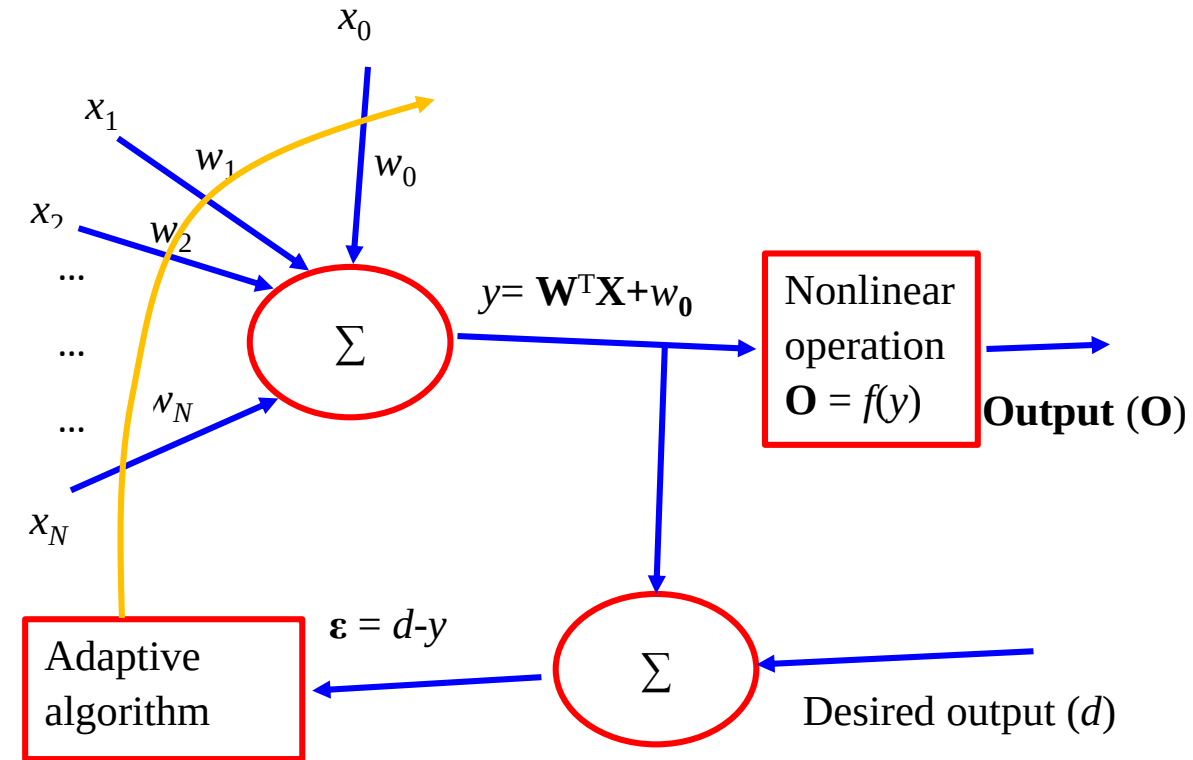
Bipolar Output $O = f(y)$



$$y = x_1 w_1 + x_2 w_2 + \dots + x_N w_N + w_0$$

$$= [x_1 \ x_2 \ \dots \ x_N] \cdot \begin{bmatrix} w_1 \\ w_2 \\ \vdots \\ w_N \end{bmatrix} + w_0 = [1 \ x_1 \ x_2 \ \dots \ x_N] \cdot \begin{bmatrix} w_0 \\ w_1 \\ w_2 \\ \vdots \\ w_N \end{bmatrix} = \mathbf{w}^T \mathbf{x}$$

- ✓ The **perceptron** only adjusts the weighting factors when the output is incorrect but **ADALINE** adjusts the weighting factors using supervised learning techniques to minimize the difference between output y and some desired output d called **error signal ϵ** .
- ✓ For example, we can use adaptive algorithm like: Least Mean Square (LMS), Recursive least squares (RLS), Frequency-Domain Adaptive Filter (FDAF) to minimize the error signal.



- ✓ Although it is not the direct building block of a neural network but the concept of ADALINE is useful in the study of ANN. Fig. above shows the basic architecture of an **ADALINE including supervised learning**.

LMS algorithm is the simplest adaptive algorithm for updating the coefficients of a digital filter. The LMS algorithm can be described as follows:

Parameters:

μ = step size

k = sampling instant

Computation:

- ✓ for $i = 0, 1, \dots, N$, initialize the weights $w_k(i)$
- ✓ calculate the output of the linear combiner filter $\hat{n} = \sum_{i=0}^N w_k(i)x_k(i)$
- ✓ calculate the error, $\varepsilon_k = d_k - \hat{n}$
- ✓ update filter weights for next iteration, $w_{k+1}(i) = w_k(i) + 2\mu\varepsilon_k x_k(i)$
- ✓ if error not minimized repeat above steps.
- ✓ else stop.

Example-1

Consider $w_2 = w_1 = w_0 = 0.5$ and bipolar binary pulse $x_1 = 1, -1$ and $x_2 = 1, -1$. Determine output O for all combination of input and comment on the result.

Taking $x_1 = x_2 = 1$, $y = x_2 w_2 + x_1 w_1 + w_0$, since $x_0 = 1$

Or, $y = 1*0.5 + 1*0.5 + 0.5 = 1.5$, $O = f(1.5) = 1$

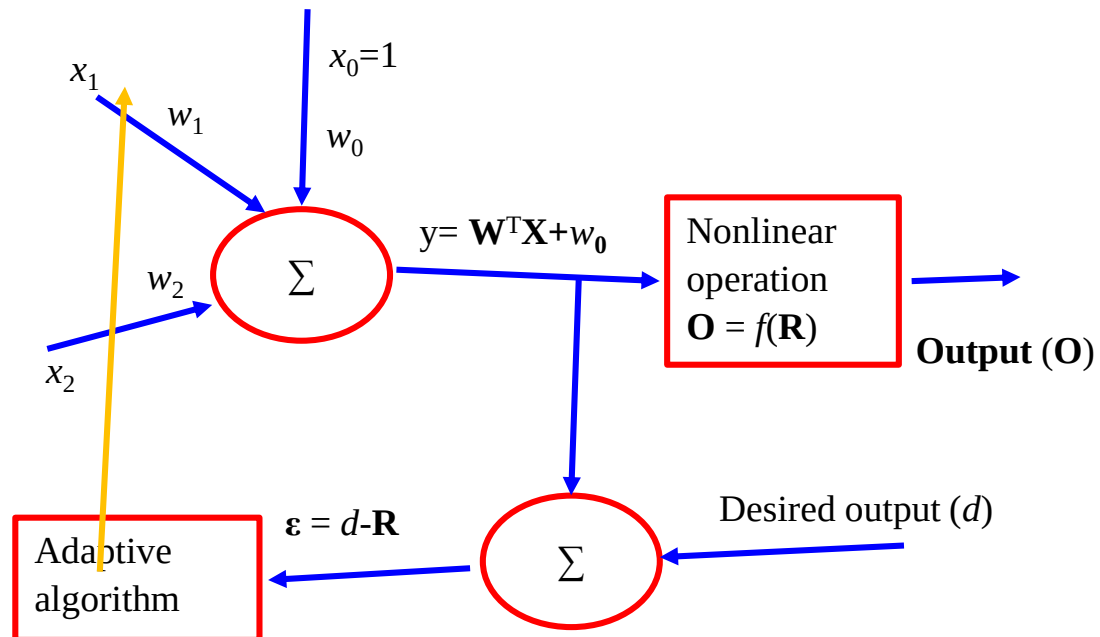
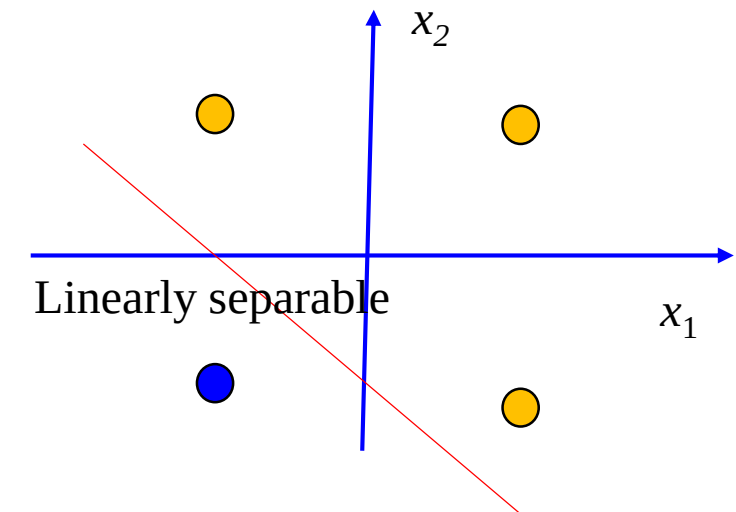
For, $x_1 = 1$, $x_2 = -1$, $y = 1*0.5 - 1*0.5 + 0.5 = 0.5$, $O = f(0.5) = 1$

For, $x_1 = -1$, $x_2 = 1$, $y = -1*0.5 + 1*0.5 + 0.5 = 0.5$, $O = f(0.5) = 1$

For, $x_1 = -1$, $x_2 = -1$, $y = -1*0.5 - 1*0.5 + 0.5 = -0.5$, $O = f(-0.5) = -1$

Let us now draw the truth table of above results.

x_1	x_2	O
1	1	1
1	-1	1
-1	1	1
-1	-1	-1



The ADALINE works as an OR gate.

Example-2

Consider $w_2 = w_1 = 0.5$ and $w_0 = -0.5$ and bipolar binary pulse $x_1 = 1, -1$ and $x_2 = 1, -1$. Determine output O for all combination of input and comment on the result.

Taking $x_1 = x_2 = 1$, $y = x_2 w_2 + x_1 w_1 + w_0$, since $x_0 = 1$

Or, $y = 1*0.5 + 1*0.5 - 0.5 = 0.5$, $O = f(1.5) = 1$

For, $x_1 = 1$, $x_2 = -1$, $y = 1*0.5 - 1*0.5 - 0.5 = -0.5$, $O = f(-0.5) = -1$

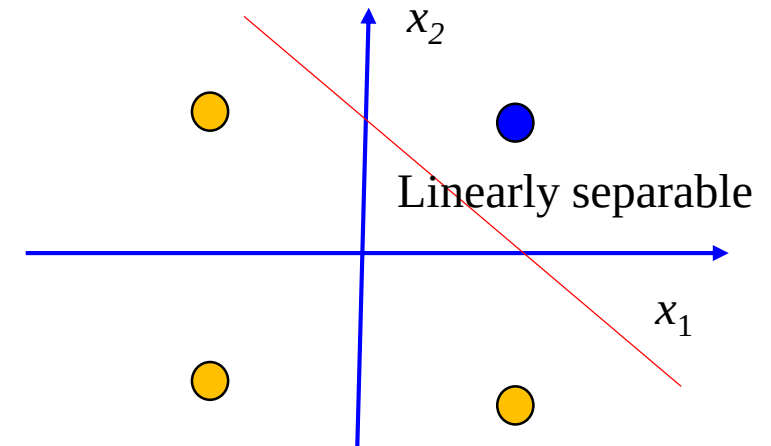
For, $x_1 = -1$, $x_2 = 1$, $y = -1*0.5 + 1*0.5 - 0.5 = -0.5$, $O = f(-0.5) = -1$

For, $x_1 = -1$, $x_2 = -1$, $y = -1*0.5 - 1*0.5 - 0.5 = -1.5$, $O = f(-1.5) = -1$

Let us now draw the truth table of above results.

x_1	x_2	O
1	1	1
1	-1	-1
-1	1	-1
-1	-1	-1

The ADALINE works as an AND gate.

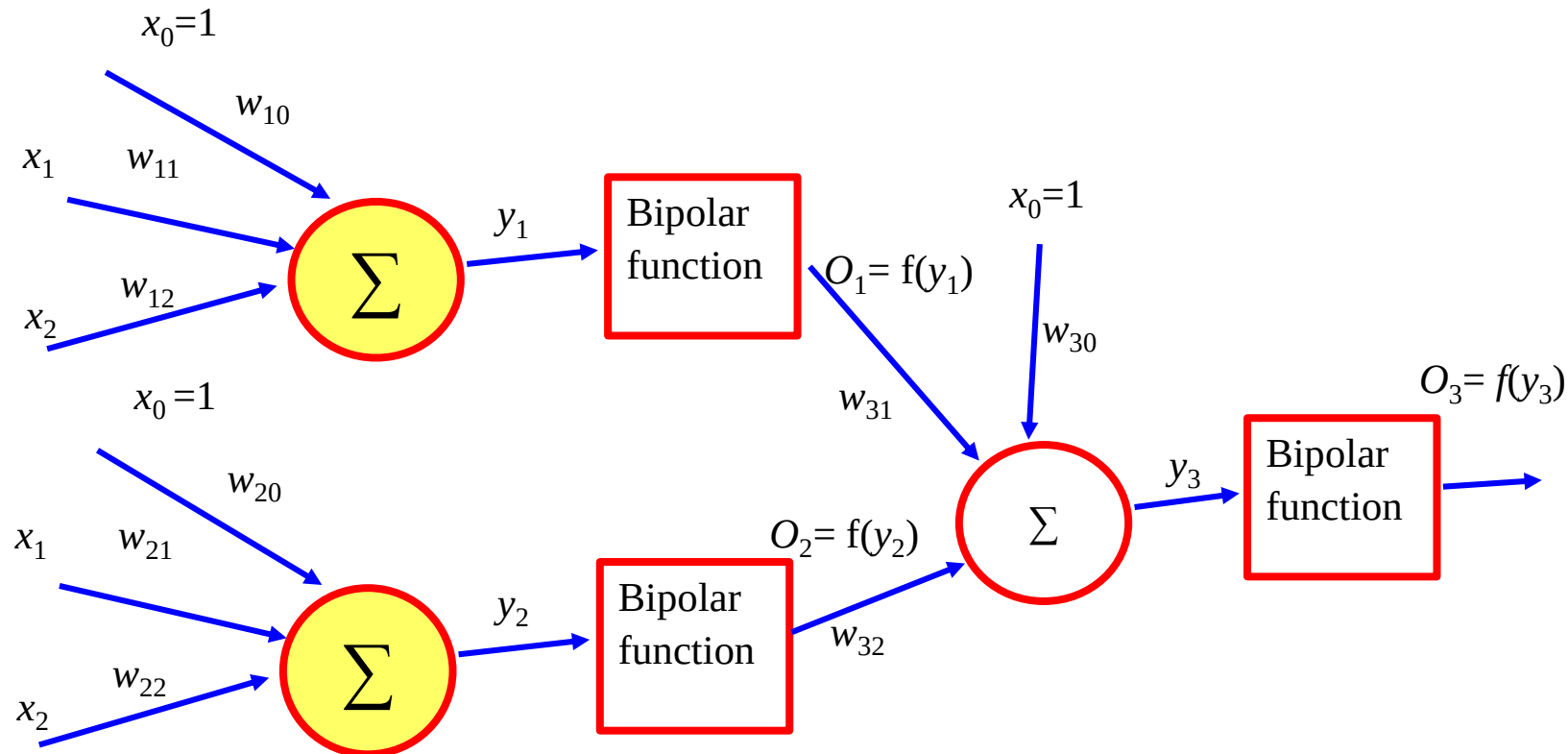


Perception vs. Adaline

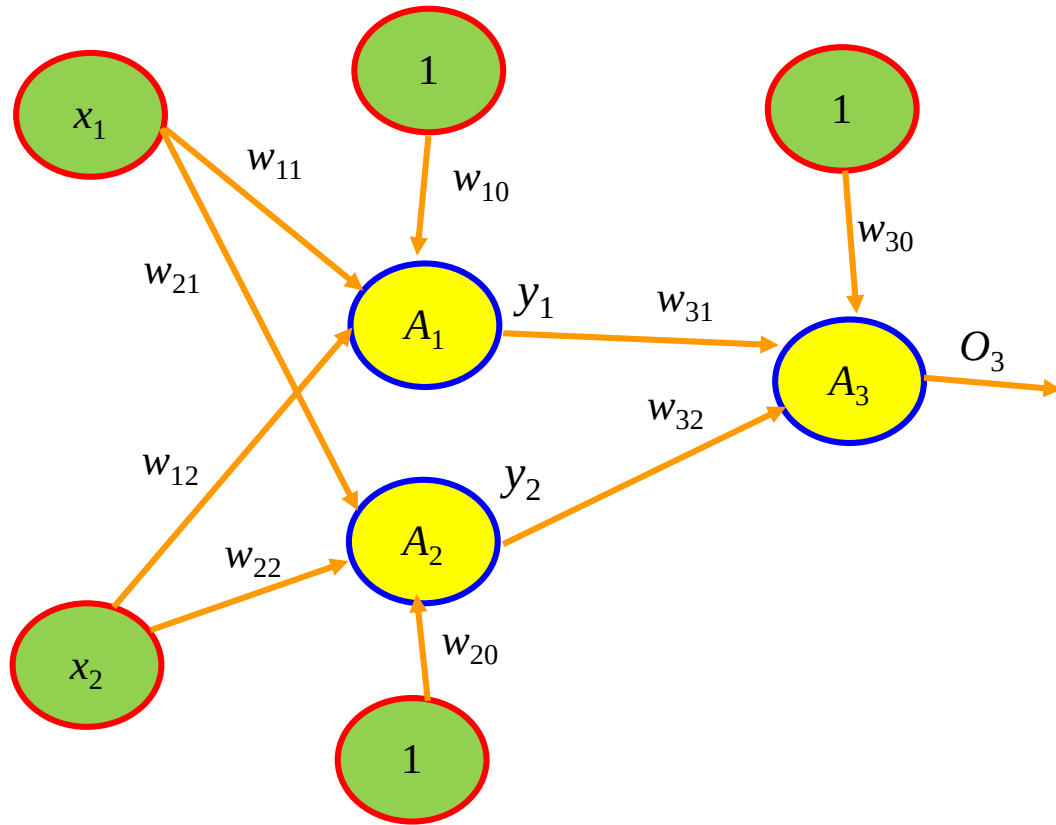
- ✓ Although both perception and Adaline use binary classification with linear decision boundary but they have some difference. The Perceptron takes input vectors from the **table** with their target values i.e. its provides an offline operation.
- ✓ On the other hand, Adaline takes **continuous input vector** from other system and compare its own output to the desired out (output of the teacher not the target value of table) to judge how far it is from optimum condition. The ADALINE can provide realtime operation on the input signal.
- ✓ The **perceptron** only adjusts the weighting factors when the output is incorrect but **ADALINE** adjusts the weighting factors using supervised learning until the **error signal ϵ** falls below the threshold.

Basic of MADALINE

An ADALINE individually cannot work as an XOR gate. In this case we have to combine two ADALINES together. Several ADALINES are combined to form an **MADALINE** or **many ADALINE** is used to segregate nonlinearly separable data points. Fig.2.3 shows the MADALINE which consists of combination of three ADALINES.



Above MADALINE can be drawn in a simpler way shown in fig. below, where bipolar function is kept hidden. Here A_1 , A_2 and A_3 are the three ADALINEs, $[x_1 \ x_2]$ is the input vector and w_{ij} ($i = 1, 2, 3$ and $j = 0, 1, 2$) are the weighting factors.



The relation between input vector and final output can be expressed by three equations like,

$$y_1 = x_1 w_{11} + x_2 w_{12} + w_{10}, O_1 = f(y_1)$$

$$y_2 = x_1 w_{21} + x_2 w_{22} + w_{20}, O_2 = f(y_2)$$

$$y_3 = O_1 w_{31} + O_2 w_{32} + w_{30}, O_3 = f(y_3).$$

Example-3

Taking,

$$w_{11} = -0.75, w_{12} = 1.3, w_{10} = -1$$

$$w_{21} = 1.5, w_{22} = -1.35, w_{20} = -1$$

$$w_{31} = 0.5, w_{32} = 0.5, w_{30} = 0.5$$

Show that MADALINE of fig. works as an XOR gate.

Ans.

Considering, $x_1=1, x_2=1$

$$y_1 = -0.75*1 + 1.3*1 - 1 = -0.45, O_1 = -1$$

$$y_2 = 1.5*1 - 1.35*1 - 1 = -0.85, O_2 = -1$$

$$y_3 = 0.5*(-1) + 0.5*(-1) + 0.5 = -0.5, O_3 = -1$$

$x_1=-1, x_2=1$

$$y_1 = -0.75*(-1) + 1.3*1 - 1 = 1.05, O_1 = +1$$

$$y_2 = 1.5*(-1) - 1.35*1 - 1 = -3.85, O_2 = -1$$

$$y_3 = 0.5*(+1) + 0.5*(-1) + 0.5 = 0.5, O_3 = +1$$

$x_1=1, x_2=-1$

$$y_1 = -0.75*1 + 1.3*(-1) - 1 = -3.05, O_1 = -1$$

$$y_2 = 1.5*1 - 1.35*(-1) - 1 = 1.85, O_2 = +1$$

$$y_3 = 0.5*(1) + 0.5*(+1) + 0.5 = 0.5, O_3 = +1$$

$x_1=-1, x_2=-1$

$$y_1 = -0.75*(-1) + 1.3*(-1) - 1 = -1.55, O_1 = -1$$

$$y_2 = 1.5*(-1) - 1.35*(-1) - 1 = -1.15, O_2 = -1$$

$$y_3 = 0.5*(-1) + 0.5*(-1) + 0.5 = -0.5, O_3 = -1$$

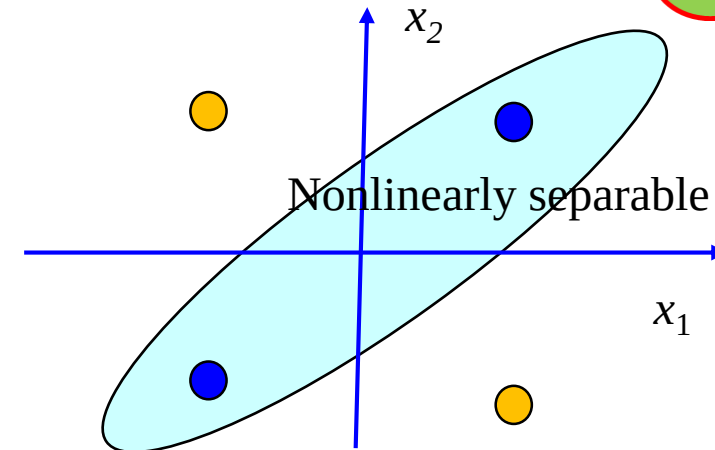
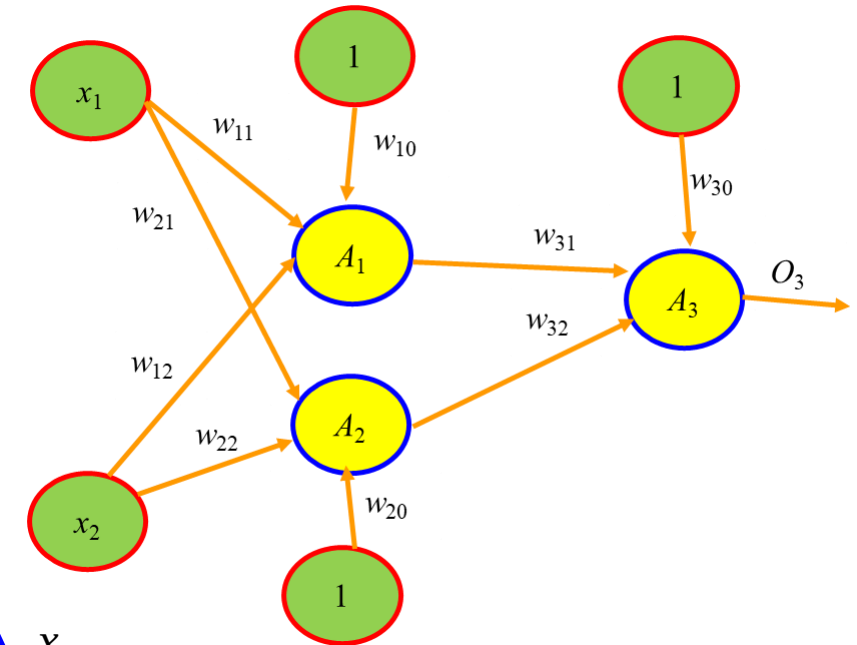
Let us now draw the truth table of above results.

x_1	x_2	O
1	1	-1
1	-1	1
-1	1	1
-1	-1	-1

$$w_{11} = -0.75, w_{12} = 1.3, w_{10} = -1$$

$$w_{21} = 1.5, w_{22} = -1.35, w_{20} = -1$$

$$w_{31} = 0.5, w_{32} = 0.5, w_{30} = 0.5$$



The MADALINE works as an XOR gate.